



Universidad
Nacional
de Quilmes

A la caza de las vinchucas

Materia: Programación con Objetos II

Alumnos: Flamini Guido - Gómez Leandro Nehuén

Docentes: Diego Cano - Matias Butti - Diego Torres

E-mails: gflamini2000@gmail.com - lgomez.nehuen@gmail.com

Decisiones de Diseño:

Comenzamos con el desarrollo de un diagrama UML donde intentábamos buscar la responsabilidad única de cada clase para poder definirla y de esta forma poder aplicar el Principio de Separación de Responsabilidades viendo como se tendrían que comportar cada una y como estas deberían relacionarse.

Usuario

La clase Usuario implementa un patrón State para actualizar su categoría cada vez que necesite hacerlo. Cada opinión que emite y cada muestra que envía son registrados para luego ejecutar el algoritmo que se encargará de recategorizarlo. El comportamiento del Usuario se verá condicionado por su estado. Por ejemplo, un Usuario Básico, al ejecutar el método “opinarSobre” en una muestra que tenga opiniones de expertos, lanzará una excepción. Los roles según Gamma et al. son:

- **Context:** Usuario
- **State:** UsuarioState
- **ConcreteState:** UsuarioBasico y UsuarioExperto

Los especialistas comienzan siendo Expertos y nunca cambian su estado.

Ubicación y ZonaDeCobertura

Una Ubicación, además de tener una longitud y latitud, es capaz de calcular la distancia entre ella misma y otras ubicaciones. A su vez, conforma el epicentro de una zona de cobertura, que son capaces de saber con qué otras zonas se solapan y qué muestras se encuentran dentro de su área. Las zonas de cobertura se agrupan en una clase llamada GestorDeZonas.

Opinion

Los usuarios emiten opiniones(votaciones) que son almacenadas en una clase Muestra. Hubo un breve debate con respecto a cómo representar los tipos de insecto que contiene la clase Opinion. Se decidió que la manera más conveniente de hacerlo era con un enum, **TipoDeOpinion**, dado que de esta manera se evita cualquier tipo de error asociado a una solución con Strings.

Muestras

Las muestras, elemento principal del proyecto, son agrupadas dentro de una clase **GestorDeMuestras**. Se asume que el sistema Web recibe las muestras luego de que estas sean tomadas por una persona utilizando la aplicación móvil.

Cuando se registra una muestra en el Gestor, éste notifica al Usuario creador de la misma para que la registre dentro de sus muestras enviadas. De esta manera, Usuario tendrá la información necesaria para realizar su propia recategorización.

El GestorDeMuestras, a su vez, puede iniciar una **búsqueda de muestras** con los siguientes filtros: FiltroFechaDeCreacion, FiltroNivelDeValidacion, FiltroTipoDeInsecto, FiltroFechaDeVotacion, FiltroAND y FiltroOR. Estos dos últimos son filtros compuestos, es decir, pueden tener varios filtros dentro suyo. Por ejemplo, Si FiltroAND tiene un FiltroFechaDeCreacion y un FiltroNivelDeValidacion, va a retornar el resultado de hacer la operación lógica “FiltroFechaDeCreacion.cumple(Muestra) && FiltroNivelDeValidacion.cumple(Muestra)”. Los filtros fueron desarrollados con el patrón **Composite**. Los roles son los siguientes:

- **Cliente:** BusquedaDeMuestras
- **Component:** FiltroMuestra(Clase abstracta)
- **Leaf:** Los filtros simples, es decir, FiltroFechaDeCreacion, FiltroNivelDeValidacion, FiltroTipoDeInsecto y FiltroFechaDeVotacion
- **Composite:** FiltroOR y FiltroAND

Se asume que cuando se agregan los filtros FiltroFechaDeCreacion y FiltroFechaDeVotacion a la búsqueda se desea obtener las muestras que hayan sido creadas u opinadas después de la fecha indicada.

Para definir el estado de una Muestra se utiliza el patrón **State**:

- **Context:** Muestra
- **State:** MuestraState
- **ConcreteState:** MuestraVerificada, MuestraNoVerificada, MuestraParcialmenteVerificada.

Organización

En nuestra solución, las funcionalidades externas serían implementadas con un patrón Strategy. En este modelo, según Gamma et al, la interfaz **FuncionalidadExterna** tomaría el

rol de **Strategy**, mientras que la **Organización** tomaría el rol de **Contexto**. Las diferentes funcionalidades que implementen la interfaz serían **ConcreteStrategy**.

Además, aplicamos el patrón *Observer*, donde la clase **Organizacion(ConcreteObserver)** implementa la *interfaz* **ObserverDeOrganizacion(Observer)**. La clase **ZonaDeCobertura** actúa como **ConcreteSubject** mediante la implementación de la interfaz **SubjectZonaDeCobertura(Subject)**, notificando automáticamente cuando se crea o se verifica una muestra.

La zona de cobertura, para notificar a las organizaciones de la verificación de una muestra, debe ser notificada a su vez por la clase Muestra. Por esta razón, además de actuar como **ConcreteSubject** para Organización, actúa también como **ConcreteObserver** para una Muestra. En esta segunda aplicación los roles quedarían de la siguiente manera:

- **Observer:** ObserverDeZonaDeCobertura(Interfaz)
- **ConcreteObserver :** ZonaDeCobertura
- **Subject:** SubjectMuestra(Interfaz)
- **ConcreteSubject:** Muestra